GUIA VISUAL 101

COMPILAÇÃO, TOOLCHAINS E O ECOSSISTEMA DE DEV

O README diz "just run make". O `make` *responde* `openssl/ssl.h: No such file` *. Meia hora depois,* `undefined reference to 'pcap_open_live'` *. Depois disso o binário compila, roda e morre em* `cannot open shared object` *.* ***Três erros, três etapas, três soluções sem nada a ver uma com a outra.***

101
GUIAS LINUX

AS QUATRO ETAPAS QUE UM GCC ESCONDE — E ONDE CADA ERRO NASCE



Os três erros da abertura, decodificados: 1) `ssl.h: No such file` nasce no **pré-processador** — falta o cabeçalho, que vem no pacote `-dev` (`libssl-dev`); o `apt-file search openssl/ssl.h` vai do arquivo ao pacote. **2)** `undefined reference` nasce no **ligador** — o cabeçalho existia e o símbolo foi declarado, mas ninguém disse onde está a implementação: falta `-lpcap`. **3)** `cannot open shared object` nasce **depois de tudo**, na carga do programa — a biblioteca existia na compilação e não está no caminho de busca em execução. **Rode as quatro etapas isoladas uma vez na vida:** `gcc -E prog.c` transforma 4 linhas em ~800 — é a demonstração mais direta de que `#include` é inclusão *textual*.

ONDE O CARREGADOR PROCURA — E O QUE RESPONDE A CADA PERGUNTA SOBRE UM ELF

1 `LD_PRELOAD` — carregado antes de tudo, sobrepõe símbolos

2 `RPATH / RUNPATH` — caminho gravado dentro do próprio ELF

3 `LD_LIBRARY_PATH` — variável de ambiente da sessão

4 `cache de /etc/ld.so.conf` — `ldconfig -p`

5 `/lib e /usr/lib` — os diretórios padrão, por último

↓ nenhum dos cinco tinha → `cannot open shared object`

A ordem importa e é explorável: quem controla `LD_PRELOAD` ou um diretório de `RPATH` gravável substitui qualquer função de biblioteca de um programa alheio — a técnica por trás de instrumentação legítima *e* de uma classe inteira de escalada de privilégio. **É por isso que o carregador ignora `LD_PRELOAD` e `LD_LIBRARY_PATH` em binário SUID.**

PERGUNTA SOBRE O BINÁRIO	COMANDO
Executável, biblioteca ou objeto? Estático ou dinâmico?	`file`
De quais bibliotecas depende?	`ldd` · `readelf -d`
Qual símbolo está faltando?	`nm -D --undefined-only`
Tem símbolos de depuração?	`readelf -S | grep debug`
Qual é o código de verdade?	`objdump -d` · `strings`

É o mesmo conjunto da engenharia reversa — muda a intenção, não a ferramenta. **Estático × dinâmico:** `-static` embute tudo (~800 KB contra ~16 KB) e corrigir a biblioteca obriga a recompilar — em troca, **roda em quase qualquer Linux da mesma arquitetura**: a escolha certa para a ferramenta que tem de funcionar onde você não pode instalar nada.

NA PRÁTICA — RECONHECER O SISTEMA DE CONSTRUÇÃO E ACHAR OS FLAGS CERTOS

kali@kali: ~/fonte

```
# do cabecalho que falta ao pacote que o traz:
$ apt-file search openssl/ssl.h
libssl-dev: /usr/include/openssl/ssl.h
# os flags certos, sem decorar nome de biblioteca:
$ pkg-config --cflags --libs openssl
-I/usr/include/openssl -lssl -lcrypto
$ gcc prog.c $(pkg-config --cflags --libs libpcap) -o prog
# registrar um diretorio de biblioteca de vez:
$ echo /opt/lib | sudo tee /etc/ld.so.conf.d/minha.conf
$ sudo ldconfig   # reconstroi /etc/ld.so.cache
# compilacao cruzada: ARM e Windows, do proprio PC
$ aarch64-linux-gnu-gcc prog.c -o prog-arm64 -static
$ qemu-aarch64-static ./prog-arm64   # e roda aqui
$ x86_64-w64-mingw32-gcc prog.c -o prog.exe
# -O0 -g para depurar; -O2 na entrega. E por padrao:
$ gcc -Wall -Wextra -Werror -fstack-protector-strong \
      -D_FORTIFY_SOURCE=2 -O2 prog.c
```

ARQUIVO NO REPOSITÓRIO	A SEQUÊNCIA
Makefile	`make -j$(nproc)` — recompila só o que tem fonte mais novo que o objeto
configure · Makefile.am	Autotools: `./configure --prefix=/opt/x && make && make install`
CMakeLists.txt	`cmake -S . -B build && cmake --build build`
meson.build	`meson setup build && ninja -C build`

Receita do `make` se indenta com TAB, nunca com espaço — é a causa do `missing separator. Stop.`; o culpado é o editor que converte TAB.

`sudo make install` **instala fora do controle do gerenciador de pacotes.** Os arquivos não aparecem em `dpkg -L` , não somem com `apt remove` e podem **sobrescrever arquivos de pacotes**. Duas mitigações: instale sempre com `--prefix=/usr/local` ou `/opt/<nome>` , que o FHS reserva justamente para isso; ou empacote — o `checkinstall` gera um `.deb` rudimentar a partir do `make install` e devolve o software ao controle do `apt` . E confira antes se existe `make uninstall` : nem sempre existe.

RESUMO RÁPIDO

- ✓ **Compilar são quatro etapas** — pré-processador, compilador, montador, ligador. Cada uma falha de um jeito, e **a mensagem de erro diz em qual delas você está**.
- ✓ **Cabeçalho ausente = pacote** `-dev` ; `undefined reference` **= falta** `-l<lib>` na ligação; `cannot open shared object` acontece só em *execução* e não tem relação com a compilação ter dado certo.
- ✓ `file` , `ldd` , `readelf -d` , `nm -D` e `objdump -d` respondem a perguntas distintas sobre um ELF — o mesmo conjunto serve para depurar uma build e para inspecionar binário desconhecido.
- ✓ **Cinco lugares, nesta ordem:** `LD_PRELOAD` , `RPATH` / `RUNPATH` , `LD_LIBRARY_PATH` , o cache do `ldconfig` e os diretórios padrão. A ordem é o que torna `LD_PRELOAD` poderoso e perigoso.
- ✓ `pkg-config` **produz os flags certos** sem decorar nome de biblioteca; `apt-file search` **faz o caminho inverso**, do cabeçalho ausente ao pacote que o contém.
- ✓ **Estático é grande e portátil; dinâmico é pequeno e compartilhado.** Para rodar em máquina que você não controla, `-static` .

NO KALI

Compilar exploit público é rotina no laboratório, e falha quase sempre pelo mesmo punhado de motivos. Código do Exploit-DB costuma ser antigo e presumir 32 bits: `gcc -m32` exige `gcc-multilib` e `libc6-dev-i386` . **Exploit que vai executar no alvo compila-se com** `-static` : a máquina de destino raramente tem as bibliotecas certas. E se o alvo tiver kernel ou libc muito antigos, compilar **dentro de um contêiner com imagem base da mesma época** resolve o descompasso de ABI. Tudo em laboratório ou teste autorizado por escrito: invadir é crime (Lei 12.737/2012). **O** `x86_64-w64-mingw32-gcc` **já vem no Kali** e gera binário Windows em exercício autorizado — e o mesmo par serve à defesa: um PE suspeito se inspeciona com `objdump` , `strings` e `readelf` *sem executá-lo*. `LD_PRELOAD` **é conhecimento de duas mãos.** Ofensivamente, é persistência e escalada quando há SUID que respeita variável de ambiente ou `RPATH` em diretório gravável. Defensivamente: procurar `/etc/ld.so.preload` , que aplica pré-carga a *todo* processo e é sinal clássico de rootkit; conferir `RPATH` de binários privilegiados com `readelf -d` ; e não deixar diretório de biblioteca gravável por usuário comum. **Módulo de kernel é caso especial:** um `.ko` usa o sistema de construção do próprio kernel (`make -C /lib/modules/$(uname -r)/build M=$PWD modules`) e exige os cabeçalhos — a mecânica que o DKMS automatiza. **E ferramenta compilada do fonte não é atualizada pelo** `apt` : no Kali rolling o sistema se move toda semana, e o que você instalou em `/usr/local` fica congelado até parar de funcionar por incompatibilidade de biblioteca. **Registre o que compilou** nos dotfiles — nome, versão, origem, prefixo. É o item de manutenção que todo mundo descobre tarde demais. **Na RHEL:** os pacotes terminam em `-devel` , o básico vem de `dnf group install "Development Tools"` e o `apt-file` vira `dnf provides '*/ssl.h'` .